Graph-Based Vulnerability Retrieval

Master's Thesis — Technical Progress · Checkpoint 6

Lívia

TU Munich · Siemens

May 26, 2026

CVEfixes Dataset Integration: Overview

Goal

Integrate the CVEfixes database into our pipeline to obtain a large-scale knowledge base of historical vulnerability fixes. Verify that our CPG+embedding approach generalises beyond the 70-sample AutoPatch corpus.

Database Scale

CVEs 11,873 vulnerabilities

commits 12,107 fix commits

files 51,342 changed files

methods 277,948 method changes

Predominantly C code (92%) — consistent with kernel/system-level focus of our pipeline. Average patch adds 22 lines and removes 5 lines (asymmetric fixes).

Extracted Corpus Statistics

Metric	Value
Method pairs	4,607
Unique CVEs	1,777
Unique CWEs	92
Languages	C (92%), C++ (8%)
Avg lines added	22.4
Avg lines removed	5.4
Avg similarity ratio	0.563
Severity: CRITICAL	557
Severity: HIGH	1,132
Severity: MEDIUM	1,388

CVEfixes: CWE Distribution & Filtering

Top CWEs in CVEFixes

CWE	Name	Count
CWE-119	Buffer Overflow	507
CWE-125	Out-of-bounds Read	393
CWE-20	Improper Validation	308
CWE-416	Use After Free	304
CWE-476	NULL Ptr Deref	259
CWE-362	Race Condition	250
CWE-787	Out-of-bounds Write	150
CWE-190	Integer Overflow	137
CWE-400	Resource Exhaustion	137
CWE-401	Memory Leak	77

AutoPatch CWE Overlap Filter

Filtered CVEfixes to CWEs present in the AutoPatch dataset (23 CWE types) to ensure the expanded corpus covers the same vulnerability families.

Metric	Value
AutoPatch CWE types	23
CVEfixes entries matching	**1,618**
Coverage of extracted corpus	35.1%
CVE-based filter (exact CVEs)	0

ℹ Zero CVE overlap — purely temporal: CVEfixes ends July 2024; AutoPatch CVEs are from late 2024–2025. The datasets are fully complementary (same CWE families, disjoint time periods).

Preliminary CPG Study: Experiment Design

Goal

Validate that our Joern CPG generation + graph-diff pipeline works on CVEfixes data. Question: does the diff-based slicing algorithm produce meaningful vulnerability subgraphs on this new dataset?

Setup

N 10 examples, 10–50 lines each

CWEs 190, 362, 125, 285, 20, 367, 1284, 401, 264, 416

Pipeline Joern export → load CPG → compute_graph_diff

Key Result

✅ 8/10 produce valid slices. Pipeline generalises to CVEfixes without modification.

⚠️ Slices are large (60–90% of CPG). Fix-adjacent nodes dominate.

❌ 2/10 fail: very small functions (7 nodes) where Joern produces trivial graphs. These have no semantic content to slice.

Preliminary CPG Study: Results

		nodes		*unique source lines*			
#	**CVE / CWE**	**Before**	**After**	**Slice**	**Removed**	**Fix-adj**	**GT**
1	CVE-2024-22862 / CWE-190	265	281	33	3	24	2
2	CVE-2020-12652 / CWE-362	7	7	0	0	0	2
3	CVE-2018-8789 / CWE-125	106	107	9	1	9	2
4	CVE-2016-7097 / CWE-285	7	7	0	0	0	12
5	CVE-2013-7271 / CWE-20	287	275	32	21	20	5
6	CVE-2022-1974 / CWE-367	264	256	32	3	31	2
7	CVE-2022-25375 / CWE-1284	194	231	22	17	7	3
8	CVE-2019-19079 / CWE-401	151	160	15	6	12	5
9	CVE-2016-3841 / CWE-264	284	302	29	11	24	5
10	CVE-2017-16939 / CWE-416	202	121	17	14	7	8

Successes

- 8/10 produced non-empty slices
- Pipeline handles diverse CWEs

Observations

- Fix-adjacent nodes dominate slices
- Slices are 60–90% of full CPG

Failures

- 2/10 produced empty slices (7 nodes)
- Small functions → Joern parse issues

Slicing Depth Study: Experiment Design

Goal

The preliminary study showed slices are 60–90% of the full CPG — too large, diluting the vulnerability signal. Question: how much flow context around changed nodes is actually needed?

Metrics

hit@K Query's CWE family in top-K retrieved results

MRR Mean reciprocal rank of first same-CWE hit

CWE recall Fraction of CWE families covered in top-K

Hypothesis

Tighter slices (fewer context nodes) should improve embedder discrimination by concentrating the change signal.

Slice Configurations

Config	Depth	Threshold
depth_1	1-hop	none
depth_2	2-hop	none
depth_3	3-hop	none
changed_only	0	$w > 0.3$
removed_only	0	$w > 0.9$
tight_1hop	1-hop	$w > 0.5$

Split: 75 index / 17 query (5 CWE families)

Embedders: GIN, Combined, CodeBERT+Pattern

Slicing Depth Study: Results

	GIN			Combined			CodeBERT+Pattern		
Slice Config	hit@1	MRR	CWE	hit@1	MRR	CWE	hit@1	MRR	CWE
depth_1	0.0%	.020	.238	11.8%	.118	.237	11.8%	.118	.237
depth_2	0.0%	.020	.232	5.9%	.088	.242	11.8%	.118	.237
depth_3	0.0%	.020	.232	5.9%	.088	.242	11.8%	.118	.237
changed_only	5.9%	.059	.225	11.8%	.118	.227	11.8%	.118	.255
removed_only	5.9%	.059	.192	11.8%	.118	.150	11.8%	.118	.175
tight_1hop	5.9%	.059	.225	11.8%	.118	.227	11.8%	.118	.260

GIN

- Embedding collapse (sim=0.99)
- Deeper context does not help

Combined / CB+Pattern

- Slice depth has minimal impact
- Tight slices slightly improve CWE recall

Overall

- Low absolute hit@1 ($\leq$11.8%)
- Small query set limits significance

Slicing Depth Study: Key Findings

- **GIN suffers severe embedding collapse** — mean pairwise cosine similarity = 0.994 on CVEfixes (all graphs map to nearly the same point). *GIN unsuitable without retraining on diverse data.*
- **Slice depth is not the bottleneck** — depth 1–3 produce nearly identical results for CodeBERT+Pattern. The change signal is already captured at 1-hop. *Tight_1hop achieves best CWE recall (0.260).*
- **Tighter slices slightly improve CWE discrimination** — removed_only (62 nodes avg) hurts CWE recall, but tight_1hop (161 nodes) matches or beats full-depth. *Optimal: keep changed nodes + 1 hop of flow context with weight filtering.*
- **Small evaluation set (17 queries)** — results are noisy; differences below $\sim$10% are not statistically significant at this scale.

GNN Triplet Training: Experiment Design

Goal

Previous experiments show near-random clustering (NMI≈0.02). Hypothesis: supervised training with triplet loss on CWE family labels can teach the GNN to produce embeddings that separate vulnerability types.

CWE Families (5 classes)

- Memory corruption (CWE-119/120/122/125/787)
- Use-after-free (CWE-416/415/763)
- Integer issues (CWE-190/191/189/681)
- NULL ptr deref (CWE-476/824)
- Race condition (CWE-362/367/667)

Training Configuration

Parameter	Value
Graphs	695
Hidden size	128
GNN layers	4
Max nodes	200
Learning rate	5e-4
Margin	0.3
Triplets/epoch	512
Early stopping	patience=15
Epochs run	51 (stopped)

GNN Triplet Training: Results

Training Metrics

Metric	Value
Final train loss	0.169
Final val loss	0.426
Training time	27.4 min
Retrieval (train set)	
P@1	30.1%
P@3	26.2%
P@5	26.0%
P@10	24.3%
MAP	22.8%
MRR	47.7%
Val P@1	16.2%

Clustering Quality

Metric	Value
ARI	0.018
Homogeneity	0.020
NMI	0.022

Interpretation

- Train retrieval shows learning signal (MRR=0.477)
- Val P@1 drops to 16.2% — significant overfitting
- Near-zero clustering scores: embeddings do not form CWE-family clusters
- Gap between train/val loss confirms overfitting (0.17 vs 0.43)

GNN Triplet Training: Key Findings

- **Triplet loss learns partial retrieval signal** — train MRR reaches 0.477, indicating the GNN can partially separate CWE families on seen data. *But val P@1=16.2% shows poor generalization.*
- **Clustering remains near-random** — ARI=0.018, NMI=0.022 despite 51 epochs of triplet training on 695 graphs. *GNN structure alone insufficient to cluster vulnerability families.*
- **Overfitting with 695 graphs** — model memorizes training triplets (train loss 0.17) but fails on held-out data (val loss 0.43). *Need more data or stronger regularization (dropout=0.2 was used).*
- **Graph structure alone is not discriminative for CWE families** — confirms need for hybrid (graph + code text) approaches like CodeBERT+Pattern.

Token-Guided Slicing: Experiment Design

Goal

Since graph structure alone fails to discriminate CWE families, try a simpler approach: use discriminative code tokens (identified via TF-IDF + LinearSVC) to guide which CPG nodes to include in the slice. Inspired by "sink" concept from VulChecker.

Slicing Strategies

token_guided Slice to nodes matching discriminative keywords per CWE family

diff_based Standard: removed + fix-adjacent nodes (depth=3)

random Random 30% of nodes (baseline)

Evaluation

KMeans clustering (K=5 families), measured by ARI, NMI, homogeneity.

Discriminative Tokens (top per family)

Family	Keywords
integer	`int, val, scale, sma`
memory	`ptr, buf, size_t, str_len`
null_ptr	`unlock, page, dir, name`
race	`sk, struct, opt, arg`
uaf	`free, head, retval, bfqq`

Token hit rate: 100% (all 94 graphs matched $\geq$1 token)

Token-Guided Slicing: Clustering Results

Strategy	Embedder	ARI	NMI	Homogeneity	Nodes
token_guided	codebert_pattern	0.068	0.154	0.154	106
token_guided	codebert_flow_pattern	0.050	0.116	0.115	106
diff_based	gin	0.029	0.111	0.108	171
diff_based	combined_enriched	0.035	0.109	0.101	171
diff_based	codebert_pattern	0.017	0.100	0.099	171
diff_based	combined	0.026	0.096	0.091	171
random	codebert_pattern	0.019	0.125	0.113	65
random	codebert_flow	0.024	0.125	0.114	65
token_guided	gin	0.015	0.071	0.064	106
token_guided	combined	-0.001	0.057	0.057	106

Bottom line

- All NMI ≤ 0.154, all ARI ≤ 0.068 — **near-random**
- No strategy produces meaningful CWE-family clusters
- Random slicing matches or beats guided approaches

Implication

- Neither graph structure nor token heuristics are sufficient
- The embedding space does not separate CWE families
- Slicing strategy is irrelevant if the embedder cannot discriminate

Token-Guided: Diff Classification (5-Class)

TF-IDF + LinearSVC on Code Diffs

CWE Family	Prec	Rec	F1
integer_issues	45.5%	31.3%	37.0%
memory_corruption	31.3%	31.3%	31.3%
null_ptr_deref	17.9%	25.0%	20.8%
race_condition	39.1%	45.0%	41.9%
use_after_free	7.7%	5.3%	6.3%
Macro avg	28.3%	27.5%	27.5%

Overall accuracy: **27.5%** (random=20%)

Interpretation

- Text-only classification barely above random (27.5% vs 20%)
- Race conditions best classified — lock/unlock keywords distinctive
- UAF nearly undetectable from text alone (F1=6.3%)
- Top features per class are highly specific (function/variable names)
- Confirms need for **structural (graph) features** to distinguish CWE families

Joern Sink Verification: Experiment Design

Motivation

The structural embeddings fail to discriminate CWE families (NMI≈0.02–0.15). One hypothesis: the slices include too many irrelevant nodes. If we could identify the actual vulnerability *site* (the "sink"), we could slice around it and produce tighter, more discriminative subgraphs.

Goal

Test whether CWE-specific regex patterns can reliably locate the vulnerable operation in the CPG. If yes, these become anchors for a better slicing strategy.

CWE Distribution in Subset

CWE	Count
CWE-20 (Input Validation)	17
CWE-362 (Race Condition)	15
CWE-476 (NULL Ptr Deref)	14
CWE-416 (Use After Free)	12
CWE-400 (Resource Exhaustion)	9
CWE-190 (Integer Overflow)	9
CWE-787 (OOB Write)	8
Others (12 CWEs)	16

Setup

N 100 entries (stratified by CWE)

Patterns CWE-specific regex (free, deref, lock, arith, . . .)

Metric Hit rate: does any sink node overlap with the ground-truth fix?

Stratified proportional sampling, seed=42

Joern Sink Verification: Results (v1 → v2)

CWE	N	v1 Hit%	v2 Hit%	Optimisation Applied
CWE-787 (OOB Write)	8	100%	100%	—
CWE-264 (Permissions)	3	100%	100%	—
CWE-400 (Resource Exh.)	9	11%	100%	Event/perf patterns + fn-name match
CWE-401 (Memory Leak)	3	67%	100%	skb/alloc patterns + free-fix wrap
CWE-667 (Improper Lock)	1	0%	100%	Shared-state sink + wrap check
CWE-362 (Race Condition)	15	7%	87%	Shared-state sink + lock-wrap check
CWE-476 (NULL Ptr Deref)	14	79%	86%	memcpy pattern + null-check wrap
CWE-416 (Use After Free)	12	83%	83%	(residual = Joern parse failures)
CWE-20 (Input Validation)	17	82%	82%	(residual = 8-node graphs)
CWE-190 (Integer Overflow)	9	56%	56%	Type-change diffs: semantic gap
Overall	**100**	**64%**	**87%**	

↑ +23pp overall hit rate — 13 remaining misses: 11 are Joern C++ parse failures (8-node skeleton graphs)

Joern Sink Verification: The "Wrapping Fix" Insight

Problem

For "missing protection" CWEs, the fix *adds* a guard around existing code rather than modifying the vulnerable statement itself. Direct text overlap between sink nodes and changed lines fails.

Example (CWE-362 / CVE-2014-4652)

```
code_before:
  change = memcmp(&ucontrol->value, ...)
  if (change) memcpy(ue->elem_data, ...)

code_after:
  mutex_lock(&ue->card->user_ctl_lock);
  change = memcmp(&ucontrol->value, ...)
  if (change) memcpy(ue->elem_data, ...)

  mutex_unlock(&ue->card->user_ctl_lock);
```

The vulnerable ops (->) are **unchanged**; the fix **wraps** them.

Solution: Two-Phase Overlap Check

1. **Phase 1** — direct text overlap (standard)
2. **Phase 2** — if no direct hit, check:
   - Does the fix add a protective wrapper? (lock, null check, validation, refcount)
   - Does the sink's code still exist in `code_after`? (i.e. it was wrapped, not removed)

Applicable CWEs

CWE	Wrapper Pattern
362, 667	mutex_lock / spin_lock
476	NULL / IS_ERR check
416	refcount / kref
20, 400	if / assert / return
401	kfree / kfree_skb

Joern Sink Verification: Key Findings

87% of vulnerability sites are reachable via CWE-aware sink patterns — the regex keywords from `vuln_pattern.py` successfully locate the vulnerable operation in the Joern CPG for the majority of entries. *Validates using these patterns as Joern query seeds.*

"Wrapping fix" insight generalises across 6 CWE classes — CWE-362/400/401/416/476/667 all exhibit the pattern where the fix *adds protection around* existing operations. *The sink is the unprotected operation, not the protection itself.*

CWE-190 remains the hardest (56%) — integer overflow fixes change `int`→`size_t` type declarations; the CPG node text differs only in type name, which string matching cannot bridge. *Needs semantic type-equivalence check or AST-level diff.*

Joern parse failures account for 11/13 residual misses — C++ namespaces (`HPHP::`), macros (`SYSCALL_DEFINE5`), and templates produce 8-node skeleton graphs. *Not a pattern problem; limitation of Joern's C frontend.*

KB-Guided Sink Ranking: Experiment Design

Goal

Can a knowledge base of confirmed vulnerability subgraphs improve *precision* when ranking Joern-found sink candidates? Joern patterns find many candidates but most are false positives.

Protocol

Label For each entry: find ALL sink nodes, label TRUE/FALSE by overlap with ground-truth fix

Embed Extract 2-hop subgraph around each sink, embed via two approaches

Rank Leave-one-out: score = max cosine similarity to KB of known-TRUE subgraphs

Eval AUC-ROC: can this score separate TRUE from FALSE sinks?

Two Embedding Approaches

Embedder	Description
Structural (36d)	Node/edge-type histograms + code-pattern density + stats
CodeBERT (768d)	Concatenate node code, encode with pretrained CodeBERT [CLS]

Config: k-hop=2, leave-one-out, 87 verified entries
Script: `experiments/exp_pattern_matching.py`
Seed: 42 (same stratified subset)

Reproducible: `EMBEDDER={structural_36d, codebert}`

KB-Guided Sink Ranking: Results (Structural vs. CodeBERT)

Head-to-Head Comparison

Metric	Structural	CodeBERT
Embedding dim	36	768
Mean AUC-ROC	0.532	0.502
Mean Avg Precision	0.288	0.213
Precision@1	0.234	0.169
Precision@3	0.189	0.167
Precision@5	0.221	0.187
Baseline (random)	0.133	

Data: 77 entries, 5,130 sink candidates

682 TRUE (13.3%) vs 4,448 FALSE (86.7%)

Per-CWE AUC-ROC (Structural)

	CWE	AUC
✓	CWE-252 (Unchecked Return)	0.805
✓	CWE-190 (Integer Overflow)	0.696
✓	CWE-264 (Permissions)	0.667
✓	CWE-401 (Memory Leak)	0.623
✓	CWE-476 (NULL Ptr Deref)	0.618
~	CWE-400 (Resource Exh.)	0.544
~	CWE-20 (Input Validation)	0.537
×	CWE-787 (OOB Write)	0.474
×	CWE-416 (Use After Free)	0.459
×	CWE-362 (Race Condition)	0.456

CodeBERT: *all* CWEs at 0.500 (random)

KB-Guided Sink Ranking: Key Findings

- **CodeBERT (AUC=0.502) performs worse than structural (0.532)** — 768-dim semantic embeddings are perfectly random for within-function sink ranking. *True and false sinks in the same function share identical lexical context.*
- **Structural embedding captures weak topological signal for some CWEs** — CWE-252 (0.805), CWE-190 (0.696), CWE-476 (0.618) benefit from distinct graph connectivity patterns around true sinks. *CodeBERT washes out this signal into bag-of-tokens similarity.*
- **Fundamental architectural insight** — KB retrieval answers "does this code *look like* known vulnerable code?" but the real question is "is this node *unprotected* in this function's control/data flow?" *This is a program-analysis question, not a similarity-search question.*
- **Where KB retrieval can help** — at the *function level* (identifying CWE family, retrieving similar vulnerability patterns), not for within-function sink discrimination. *Sink ranking needs flow-sensitive analysis, not nearest-neighbour search.*

Summary: CVEfixes Exploration

Investigation	Key Result	Implication
Dataset extraction	4,607 pairs, 92 CWEs, 1,618 matching AutoPatch families	23× more data for training
Preliminary CPG study	8/10 successful, slices 60–90% of CPG	Pipeline works on CVEfixes
Slicing depth	Depth 1–3 equivalent; tight_1hop best CWE recall	1-hop flow context sufficient
GNN triplet training	MRR=0.477 train, P@1=16.2% val, NMI=0.022	GNN overfits; needs hybrid approach
Token-guided slicing	Best NMI=0.154 (CodeBERT+Pattern)	Token slicing helps CodeBERT
Joern sink verification	87% hit rate (v2); wrapping-fix pattern	CWE-aware sinks localize vuln sites
Text classification	27.5% accuracy (5-class, ran-	Text alone insufficient

Next Steps

- **Flow-sensitive sink scoring** — instead of KB similarity, use reachability features along REACHING_DEF/CDG edges: is the sink on a taint path without a guard? Directly encodes the program-analysis insight.
- **Function-level KB retrieval** — use KB at the granularity where it *does* work: retrieve similar vulnerable functions from CVEfixes, then apply sink-specific analysis within the retrieved context.
- **Sink-guided slicing** — use verified CWE-aware sink patterns (87% hit rate) as anchor for graph slicing. Slice from sink nodes outward along data/control flow.
- **Cross-dataset retrieval** — use CVEfixes as KB and AutoPatch as queries to evaluate real-world RAG performance at the function level.
- **Address CWE-190 semantic gap** — explore AST-level type-equivalence checks or LLM-based similarity for integer-overflow type-change fixes.

Repository: `graph-rag/` `python -m experiments.exp.slicing_depth_study`